Звіт з лабораторної роботи 6

Побудова діаграм послідовності для ключових сценаріїв системи

Виконав студент групи ІПЗ-23-3
Некрасов Артур Дмитрович

2026

1 Мета та вибір сценаріїв

Мета роботи — побудувати діаграми послідовності для двох ключових сценаріїв бібліотеки, простежити кожне повідомлення до методу класу та перевірити допустимість реакцій сповіщення за моделлю ЛР5. Комплект містить сценарій A для UC12, сценарій B для UC07 і підвзаємодію C, на яку посилається A. У трьох діаграмах визначено 44 повідомлення: 19 синхронних викликів, 19 повернень і 6 асинхронних повідомлень.

UC12 завершує бронювання із ЛР1 та фоновий потік ЛР4: за помилки сервісу підтвердження зберігається й надсилається повторно. UC07 дає читачеві змогу скасувати власний активний резерв і звільнити примірник. Він показує залежність поведінки від стану бронювання. Предметну область і початкові правила з ЛР1–5 збережено [1, 2].

Таблиця 1 — Відбір учасників і фрагментів

Код	Ініціатор та учасники	Межі та фрагменти
A UC12	Зовнішній планувальник, Воркер сповіщень, Сповіщення. 3 лінії життя.	Старт: process(n). Кінець: SENT, DEFERRED або NOT_READY. opt, loop, alt, ref.
B UC07	Читач, Сервіс бронювань, Бронювання, Примірник, Каталог, Журнал подій. 6 ліній життя.	Старт: cancelOwn(resId). Кінець: результат операції. alt за власником/станом і commit; par після успіху.
C Частина A	Воркер, Сповіщення, тимчасова Спроба передавання, зовнішній Шлюз. 4 лінії життя.	Один send(), один результат, закриття тимчасової спроби. alt: відповідь або тайм-аут.

Переглянуто всі шість вкладень: HTML і Markdown із завданням та два приклади у PlantUML і SVG. Приклади доставки їжі використано для перевірки нотації. Власні сценарії стосуються бібліотеки; керівні класи й нові методи явно позначено в таблиці простежуваності.

2 Сценарій A запуск і підготовка

Рисунки 1–5 — справжні знімки вікна draw.io. Великі діаграми A та B показано двома послідовними фрагментами з перекриттям. У поданнях скорочено пояснювальні примітки; усі повідомлення з ідентифікаторами збережено. Повні векторні графи й початковий код містяться у файлах SVG та PlantUML.

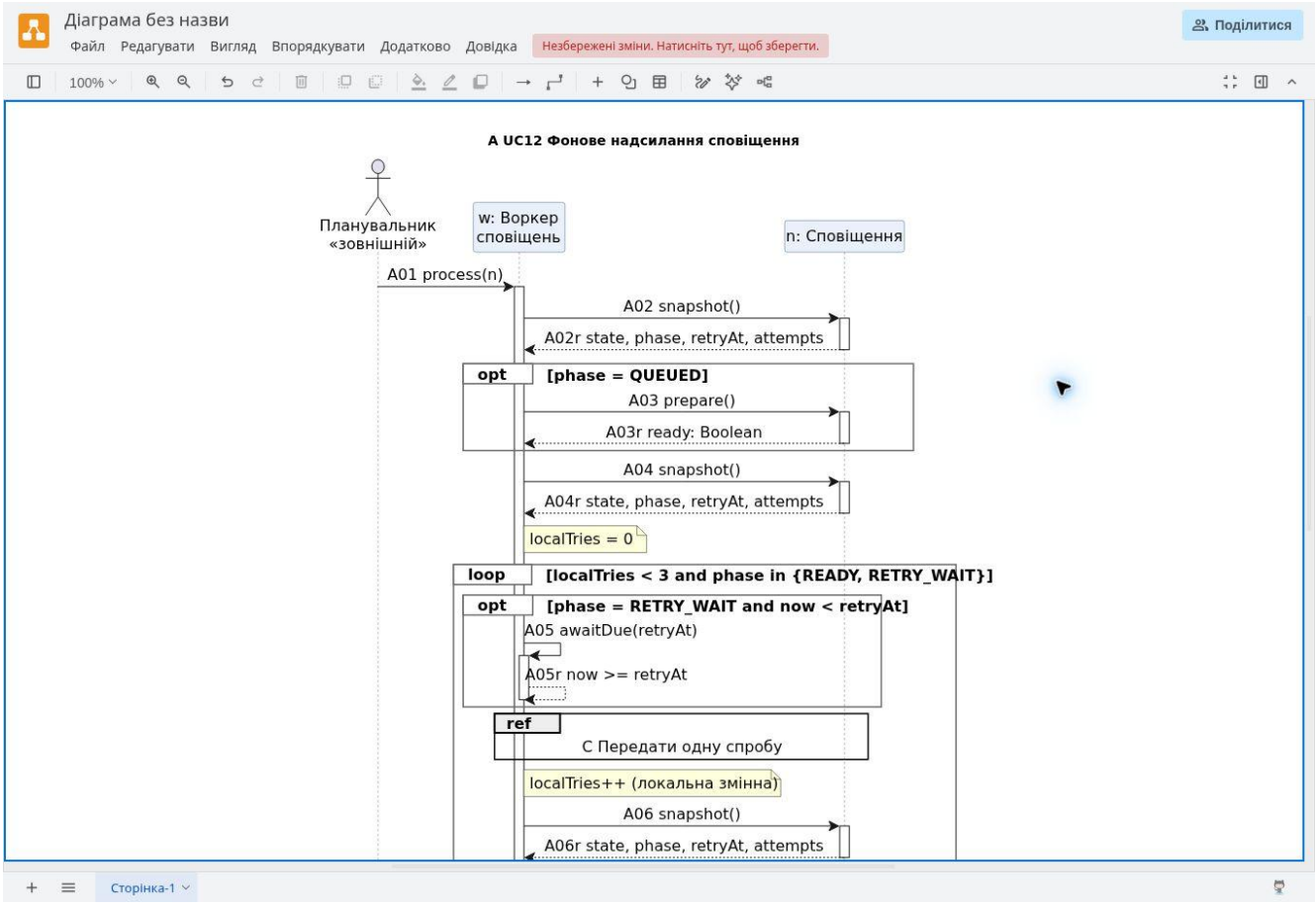


Рисунок 1 — Сценарій A від process до циклу

1. Планувальник синхронно викликає A01 process(n). Воркер отримує узгоджений знімок A02.
2. Лише для фази QUEUED блок opt виконує A03 prepare(). За коректних даних сповіщення стає READY; за хибних лишається в черзі.
3. A04 повторно читає snapshot після prepare(). Воркер установлює localTries = 0. Це усуває використання старої фази в умові циклу.

Сценарій A повтор та результат

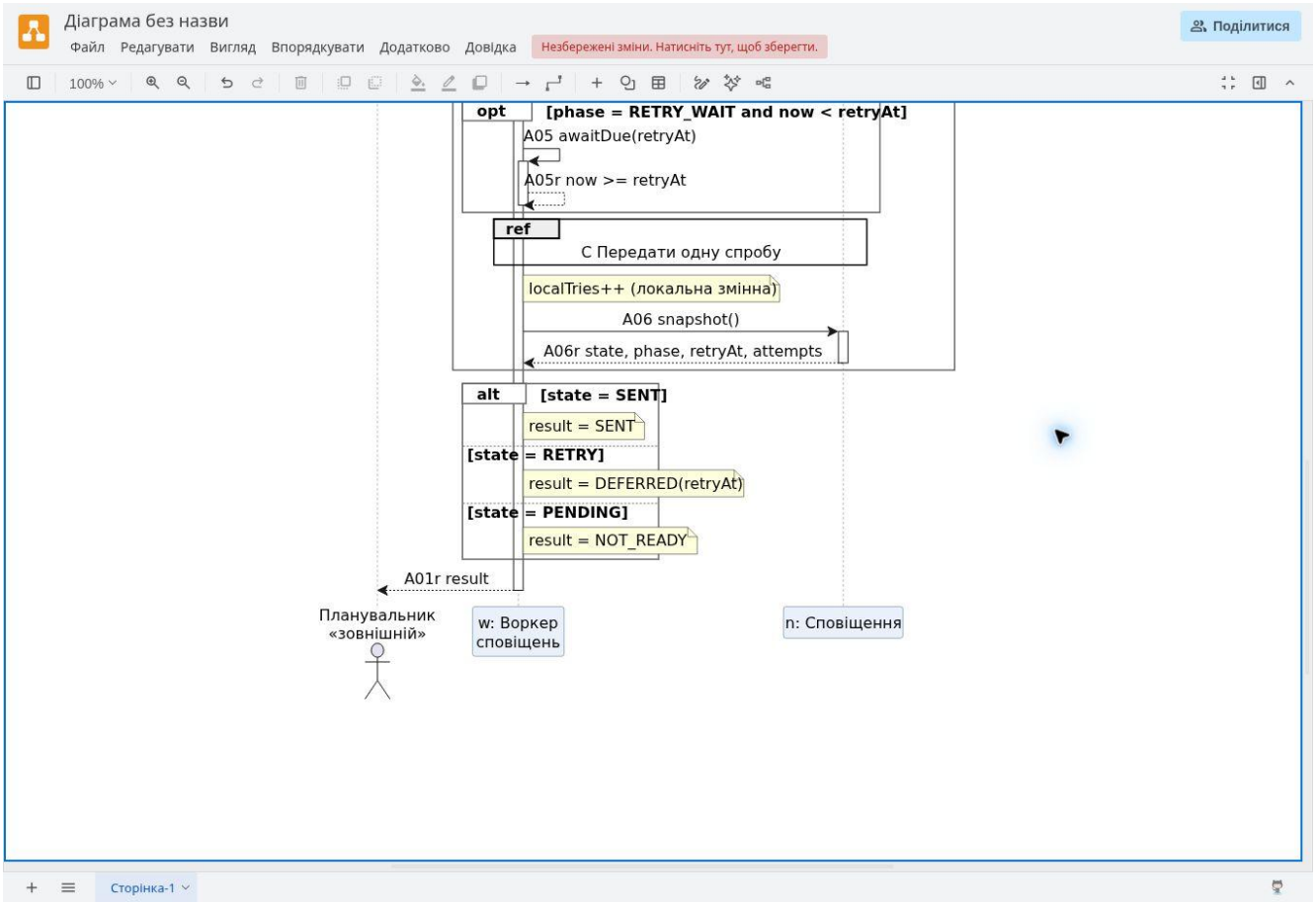


Рисунок 2 — Сценарій A цикл та завершення запуску

1. loop дозволений лише за localTries < 3 і фази READY або RETRY_WAIT. У повторі до retryAt самовиклик A05 awaitDue() очікує строк, після чого повертає поточний час.
2. ref C розгортає одну спробу. Після її результату й закриття тимчасового об’єкта localTries збільшується, A06 читає новий стан. Після SENT цикл не повторюється.
3. alt розрізняє всі значення MessageState: SENT — успіх, RETRY — DEFERRED із датою наступної спроби, PENDING — NOT_READY. A01r повертає результат планувальнику.

Бюджет 3 спроби на один запуск є проєктним уточненням ЛР6. Він не обмежує загальну кількість спроб сповіщення й не змінює його машину станів. Наступний запуск для RETRY не повторює prepare(). Для PAUSED, WAITING_RESULT або черги з хибними даними цикл не запускається. Один воркер має виключне право обробляти n; паралельна зміна його фази поза цим контекстом не допускається.

3 Підвзаємодія C одна спроба передавання

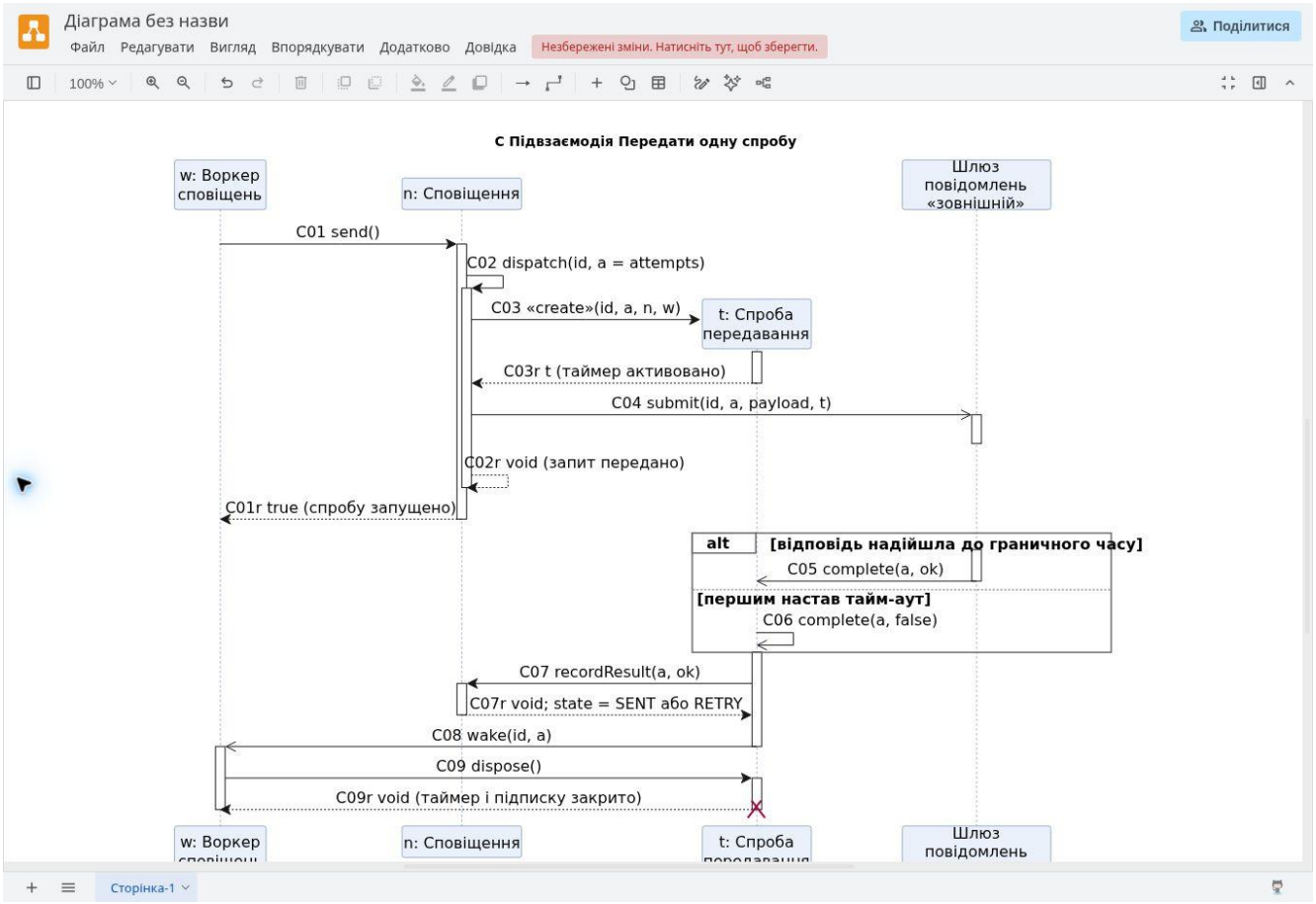


Рисунок 3 — Створення спроби, відповідь та знищення

Вхід C: READY або (RETRY_WAIT і now ≥ retryAt). C01 send() входить у Waiting: attempts збільшується, awaiting установлюється, retryAt очищується. Самовиклик C02 dispatch() створює t у C03 та асинхронно передає C04 submit() шлюзу. C02r і C01r повертаються відразу після запуску, до одержання результату.

alt обирає першу подію: C05 complete(a, ok) від шлюзу або C06 complete(a, false) за тайм-аутом. Після надходження асинхронної події спільна активація t виконує C07 recordResult(a, ok). C08 пробуджує воркер; той синхронно закриває t через C09 dispose(), отримує C09r, після чого стоїть X. Об’єкт n продовжує існувати.

t реєструється за (id, a). Реєстр транспортного адаптера пропускає лише перший результат, а dispose() закриває таймер і маршрутизацію наступних відповідей. Посилання на t у submit є логічним ідентифікатором callback, а не адресою пам’яті. Запізніла відповідь не викликає метод знищеного t. Цей контракт потребує реалізації разом із новим тимчасовим класом.

4 Сценарій B перевірка та скасування

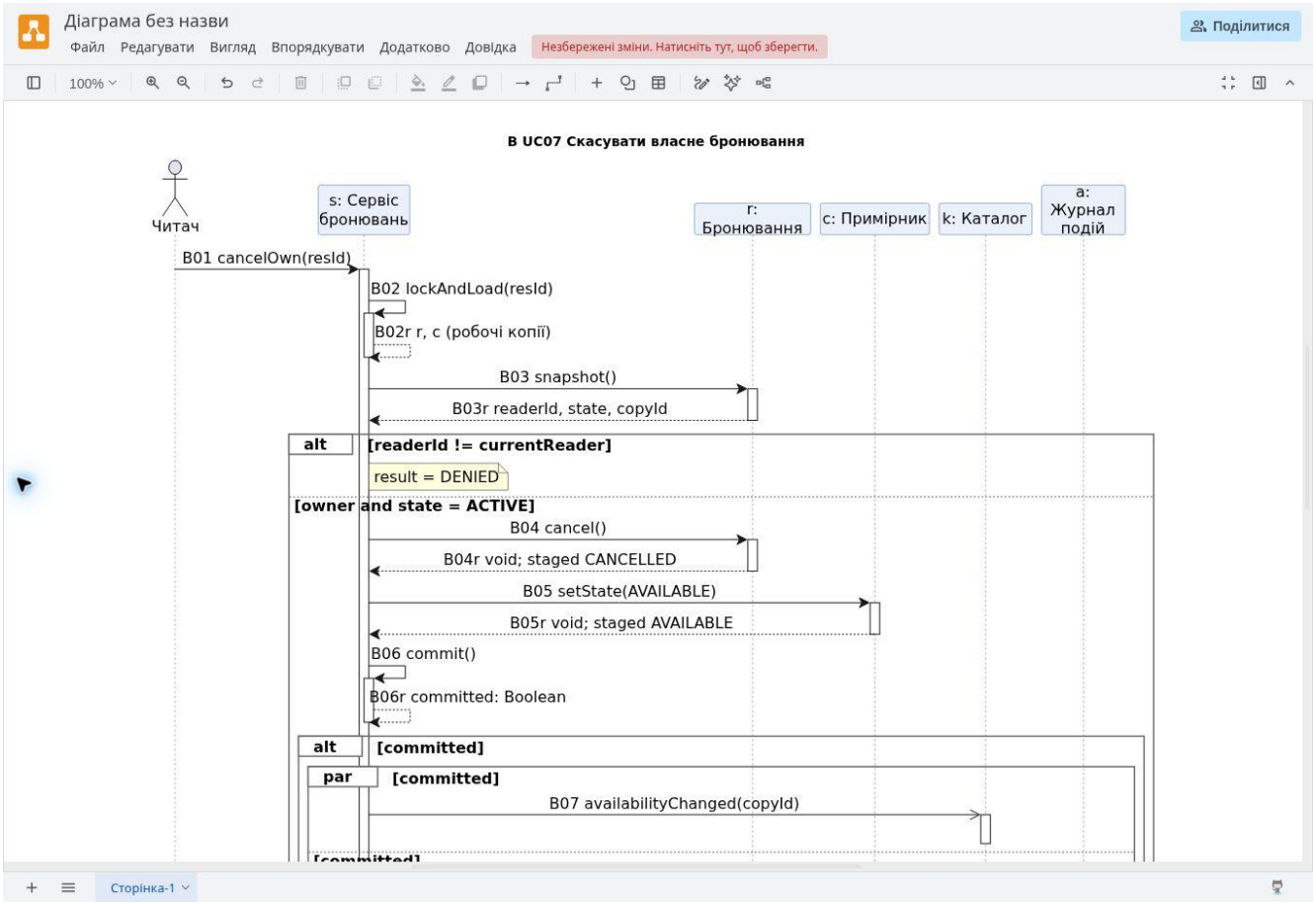


Рисунок 4 — Початок UC07 та робочі зміни

1. Авторизований читач викликає B01 cancelOwn(resId). Поточного читача сервіс визначає зі сеансу. Для цієї взаємодії резерв існує, а збережені зв’язки узгоджені.
2. B02 lockAndLoad() відкриває контекст і блокує резерв та його примірник. B03 snapshot() повертає власника, стан і copyId. Умови alt перевіряються в цьому контексті.
3. Не власник отримує DENIED. Для власного ACTIVE викликаються B04 cancel() і B05 setState(AVAILABLE). Вони змінюють робочі копії; зовнішній стан ще не опублікований.

Нормальне виконання цих доменних методів є передумовою діаграми. Технічну відмову фіксації показано окремою гілкою. Завантаження неіснуючого резерву й автентифікацію залишено за межами цього UC, щоб зосередитися на перевірці стану.

Сценарій B фіксація та паралельні дії

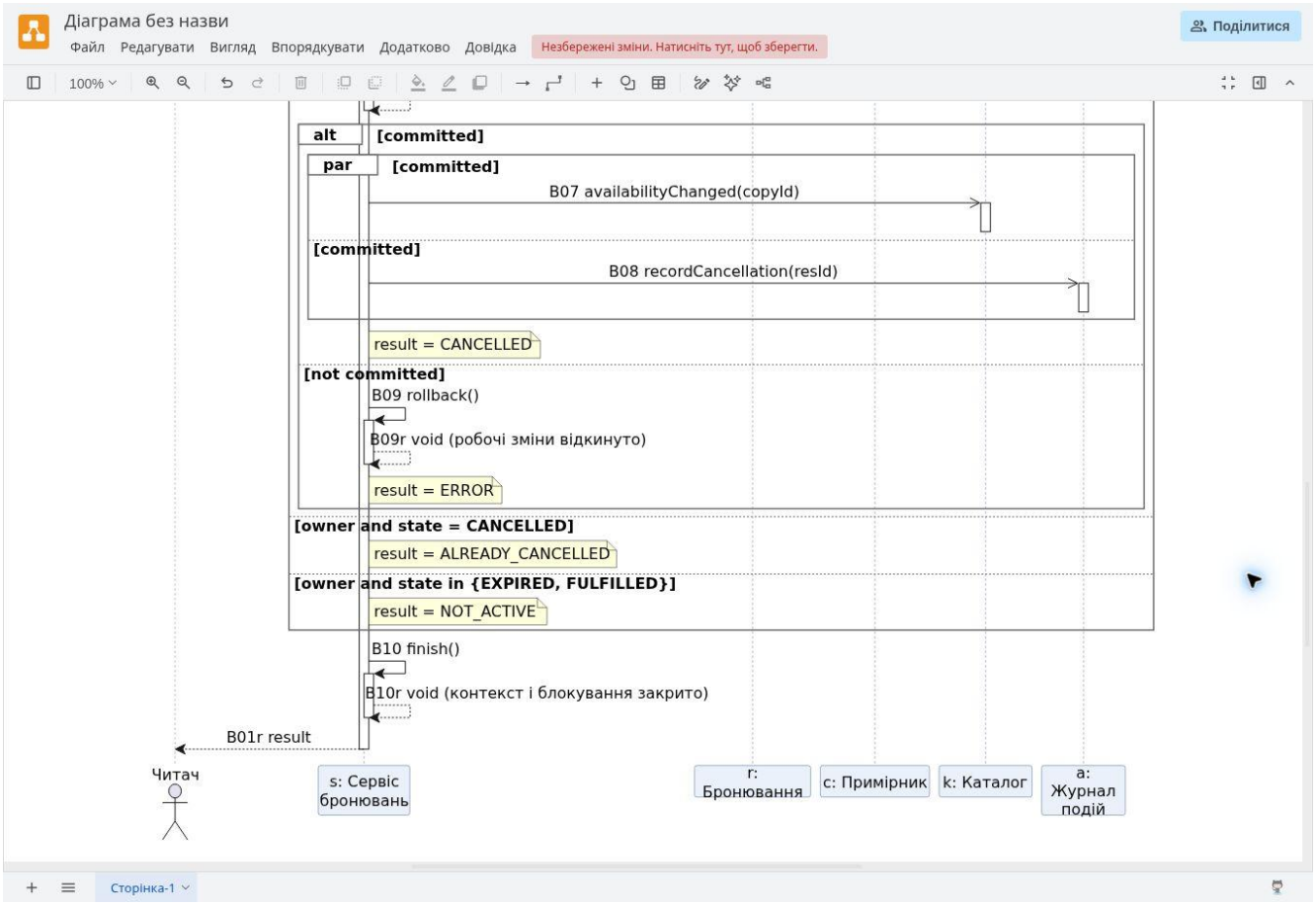


Рисунок 5 — Фіксація, незалежні гілки та повернення

B06 commit() атомарно публікує CANCELLED та AVAILABLE. Лише за committed блок par передає незалежні події B07 каталогу й B08 журналу. Обидві умови committed істинні; порядок гілок не заданий. Вони використовують уже збережені ідентифікатори та не змінюють стан бронювання. Закінчення par не означає мережевого підтвердження завершення асинхронних обробників.

За not committed B09 rollback() відкидає робочі зміни й повертається ERROR. Для власного CANCELLED результат ALREADY_CANCELLED не породжує повторних змін або подій. Для EXPIRED і FULFILLED повертається NOT_ACTIVE; виданий примірник не звільняється. B10 finish() закриває контекст на кожній гілці, потім B01r повертає результат читачу.

par не використано для залежних дій cancel(), setState() та commit(). Скасування не очікує роботи каталогу/журналу; їхня доставка й повтори є окремим інтеграційним завданням. Робота моделює взаємодію та не заявляє фактичного виконання транзакцій у БД.

5 Простежуваність повідомлень сценарію A

Кожен ідентифікатор Axx, Bxx, Cxx належить виклику; суфікс r позначає повернення цього самого виклику. Повернення не є новим методом отримувача. Асинхронні повідомлення не мають штучного синхронного повернення. Групування однакових методів у таблиці зберігає всі 44 ідентифікатори.

Таблиця 2 — Повідомлення сценарію A та їхні повернення

ID	Отримувач і метод	Походження та висновок
A01 A01r	NotificationWorker process(n: Notification): ProcessResult	потребує додавання Керівний клас воркера відсутній у ЛР2.
A02 A02r A04 A04r A06 A06r	Notification snapshot(): NotificationSnapshot	потребує додавання Запит без зміни стану; повертає state, /phase, retryAt, attempts.
A03 A03r	Notification prepare(): Boolean	уточнення ЛР5 Додано у фрагменті класу ЛР5; у початковій ЛР2 відсутній.
A05 A05r	NotificationWorker awaitDue(at: DateTime): DateTime	потребує додавання Самовиклик очікування строку без зайнятого циклу.

Статус методів відносно попередніх робіт

Початкова ЛР2 містила для Сповіщення лише send() і scheduleRetry(). У ЛР5 до уточненого фрагмента додано prepare(), recordResult(), pause(), resume() та приватний dispatch(). Тому prepare і dispatch не позначені як початкові методи ЛР2. snapshot() і керівний клас NotificationWorker потребують додавання.

snapshot() — запит, який не змінює стан і повертає узгоджені state, /phase, retryAt та attempts. Він не є новою подією життєвого циклу. awaitDue(at) — операція воркера, яка очікує строк та повертає фактичний час без збільшення лічильника спроб. Повні сигнатури всіх контрактів збережено також у Lab6_Class_Contracts.txt.

Простежуваність підвзаємодії C

Таблиця 3 — Усі повідомлення C та їхні повернення

ID	Отримувач і метод	Походження та висновок
C01 C01r	Notification send(): Boolean	наявний у ЛР2 true означає запуск; не підтверджує SENT.
C02 C02r	Notification dispatch(id: UUID, a: Integer): Void	уточнення ЛР5 Приватна дія входу в Waiting, показана самовикликом.
C03 C03r	DeliveryAttempt «create»(id: UUID, a: Integer, n: Notification, w: NotificationWorker)	потребує додавання Тимчасовий клас; конструктор установлює таймер та кореляцію.
C04	MessageGateway submit(id: UUID, a: Integer, payload: String, callback: DeliveryAttempt): Void	потребує додавання Операція зовнішнього порту; повідомлення асинхронне.
C05 C06	DeliveryAttempt complete(a: Integer, ok: Boolean): Void	потребує додавання Зовнішня відповідь або таймер; лише перший результат допускається.
C07 C07r	Notification recordResult(a: Integer, ok: Boolean): Void	уточнення ЛР5 Установлює SENT або RETRY, перевіряючи номер спроби.
C08	NotificationWorker wake(id: UUID, a: Integer): Void	потребує додавання Асинхронне пробудження воркера після результату.
C09 C09r	DeliveryAttempt dispose(): Void	потребує додавання Завершення тимчасового об’єкта, після якого стоїть X.

Для C03 задекларовано конструктор нового DeliveryAttempt. C05 і C06 викликають один complete(a, ok) з різних джерел. Знищення X є завершенням лінії життя після dispose(), а не незадекларованим прикладним повідомленням. Транспортний шлюз є зовнішнім учасником; його операцію submit потрібно включити як порт взаємодії.

Простежуваність сценарію B

Таблиця 4 — Усі повідомлення B та їхні повернення

ID	Отримувач і метод	Походження та висновок
B01 B01r	ReservationService cancelOwn(resId: UUID): CancelResult	потребує додавання Сеанс визначає поточного читача; параметр userId не довіряється клієнту.
B02 B02r	ReservationService lockAndLoad(resId: UUID): ReservationContext	потребує додавання Одна транзакція; зв’язаний примірник і резерв заблоковано.
B03 B03r	Reservation snapshot(): ReservationSnapshot	потребує додавання Атрибути є в ЛР2, операції читання ще немає.
B04 B04r	Reservation cancel(): Void	наявний у ЛР2 Викликається лише для власного ACTIVE у робочій копії.
B05 B05r	Copy setState(s: CopyState): Void	наявний у ЛР2 Аргумент AVAILABLE; оновлення ізольоване до commit.
B06 B06r	ReservationService commit(): Boolean	потребує додавання Атомарна публікація обох змін.
B07	Catalogue availabilityChanged(copyId: String): Void	потребує додавання Асинхронне оновлення проєкції доступності після commit.
B08	AuditLog recordCancellation(resId: UUID): Void	потребує додавання Незалежний асинхронний обробник журналу.
B09 B09r	ReservationService rollback(): Void	потребує додавання Відкинути незбережені робочі копії.
B10 B10r	ReservationService finish(): Void	потребує додавання Закрити контекст на кожній гілці перед поверненням читачу.

План уточнення ЛР2: додати NotificationWorker, DeliveryAttempt, MessageGateway, ReservationService та AuditLog; розширити Notification і Reservation запитом snapshot(), а Catalogue — availabilityChanged(copyId). Типи ProcessResult, CancelResult і знімки є значеннями контрактів. Початкову діаграму ЛР2 не перезаписано; відсутні елементи прямо позначені як такі, що потребують додавання.

6 Звірка з машиною станів ЛР5

Таблиця 5 — Реакції сповіщення в діаграмах A та C

Повідомлення або дія	Перехід чи інваріант ЛР5	Висновок
A03 prepare()	У черзі → Готове до спроби за validPayload; T02.	Узгоджено; за false цикл не стартує.
C01 send()	READY → Waiting або RETRY_WAIT → Waiting за now ≥ retryAt; T03/T11.	attempts++ лише при новій спробі.
C02 dispatch()	Дія входу Waiting; awaiting = true, retryAt = null.	Є приватним методом уточнення ЛР5.
C05 або C06	complete належить t; до n надходить C07 recordResult(a, ok).	Тайм-аут нормалізовано до ok = false.
C07 з ok = true	За a = attempts: Waiting → SENT → кінцевий вузол; T07/T12.	send(true) раніше не встановлював SENT.
C07 з ok = false	За a = attempts: Waiting → RETRY_WAIT; retryAt = now + 5 хв; T08.	Наступний send чекає цього строку.
snapshot, awaitDue, dispose	Перші дві дії не змінюють n; dispose знищує t, а не n.	Нових переходів машини n не потрібно.

Таблиця 6 — Контрольна траса помилки та успіху

Крок моделі	Фаза після події	Спроба	Хвилина
A04 snapshot	Ready	0	0
C01 send	Waiting	1	0
C07 recordResult(1, false)	Retry	1	1
C01 send	Waiting	2	6
C07 recordResult(2, true)	Final	2	7

Година початку умовна. Перша помилка на хвилині 1 задає retryAt = 6; повтор запускається рівно на хвилині 6. Зафіксована розбіжність чернетки — старий snapshot після prepare(). Її усунено додаванням A04. Відповідність тайм-ауту події recordResult(false) та відсутність повторної prepare() для RETRY перевірено окремо.

Висновок

Побудовано два сценарії та одну підвзаємодію з усіма потрібними операторами UML. Перевірка виявила й усунула застаріле читання фази; таблиці простежуваності прямо фіксують прогалини ЛР2. Взаємодія сповіщення узгоджується з ЛР5, а скасування бронювання не звільняє виданий примірник. Повні SVG успішно отримано з PlantUML.

Використані джерела

Методичні вказівки ЛР6: усі шість наданих файлів HTML, Markdown, PlantUML і SVG.

Власні ЛР1–5: UC07, UC12, класи бібліотеки та машина станів сповіщення.

PlantUML. Sequence Diagram. https://plantuml.com/sequence-diagram